i said algorithms not C code.

You're right! Let me provide the algorithms in proper algorithmic format with step-by-step procedures and examples for each topic in your PDF.[1]

1. Stacks & Queues

Stack Push Algorithm

Steps:

1. Check if top == MAX-1 (Stack Full condition)
2. If full, display "Overflow" and exit
3. Otherwise, increment top by 1
4. Store element at stack[top]

Stack Pop Algorithm

Steps:

1. Check if top == -1 (Stack Empty condition)
2. If empty, display "Underflow" and return -1
3. Otherwise, store stack[top] in temporary variable
4. Decrement top by 1
5. Return the temporary variable

Infix to Postfix Conversion Algorithm

Steps:

1. Create an empty stack for operators
2. Initialize an empty output string
3. Scan the infix expression from left to right:
    - If operand (a-z, 0-9): Append to output
    - If '(': Push to stack
    - If ')': Pop from stack and append to output until '(' is encountered
    - If operator (+,-,*,/,^):

- Pop all operators from stack with greater or equal precedence
- Append popped operators to output
- Push current operator to stack

4. Pop all remaining operators from stack and append to output
5. Return output string

Precedence: ^ > * / > + - > ([1]

Example: A+B*C-D/E

Symbol	Action	Stack	Output
A	Operand	empty	A
+	Operator	+	A
B	Operand	+	AB
*	Operator (higher precedence)	+*	AB
C	Operand	+*	ABC
-	Pop *, pop +, push -	-	ABC*+
D	Operand	-	ABC*+D
/	Operator	-/	ABC*+D
E	Operand	-/	ABC*+DE
End	Pop all	empty	ABC*+DE/-

Result: ABC*+DE/-[1]

Circular Queue Enqueue Algorithm

Steps:

1. Check if (rear+1) % MAX == front (Full condition)
2. If full, display "Queue Full" and exit
3. If front == -1 (first element), set front = 0
4. Calculate rear = (rear+1) % MAX
5. Store element at queue[rear]

Circular Queue Dequeue Algorithm

Steps:

1. Check if front == -1 (Empty condition)
2. If empty, display "Queue Empty" and exit
3. Store queue[front] in temporary variable

4. If front == rear (last element), set front = rear = -1
5. Otherwise, calculate front = (front+1) % MAX
6. Return temporary variable[1]

2. Linked Lists

Singly Linked List - Insert at Beginning Algorithm

Steps:

1. Create a new node
2. Set newNode→data = value
3. Set newNode→next = head
4. Update head = newNode

Time Complexity: O(1)[1]

Singly Linked List - Insert at End Algorithm

Steps:

1. Create a new node
2. Set newNode→data = value
3. Set newNode→next = NULL
4. If head == NULL, set head = newNode and exit
5. Otherwise, traverse: temp = head
6. While temp→next ≠ NULL, move temp = temp→next
7. Set temp→next = newNode

Time Complexity: O(n)[1]

Singly Linked List - Insert at Position k Algorithm

Steps:

1. Create a new node
2. Set newNode→data = value
3. If position == 1:
    - Set newNode→next = head
    - Update head = newNode
    - Exit
4. Otherwise, temp = head
5. Loop i from 1 to position-1:

- If temp == NULL, display "Invalid Position" and exit
- Move temp = temp→next

6. Set newNode→next = temp→next
7. Set temp→next = newNode

Time Complexity: O(n)[1]

Singly Linked List - Delete from Beginning Algorithm

Steps:

1. If head == NULL, display "List Empty" and exit
2. Store head in temp
3. Update head = head→next
4. Free temp

Time Complexity: O(1)[1]

Doubly Linked List - Insert at Middle Algorithm

Steps:

1. Create a new node with data = value
2. Set temp = head
3. Loop i from 1 to position-1:
   - If temp == NULL, exit
   - Move temp = temp→next
4. Set newNode→next = temp→next
5. Set newNode→prev = temp
6. If temp→next ≠ NULL, set temp→next→prev = newNode
7. Set temp→next = newNode[1]

Doubly Linked List - Delete from Last Algorithm

Steps:

1. If head == NULL, exit (empty list)
2. If head→next == NULL:
   - Free head
   - Set head = NULL
   - Exit
3. Set temp = head
4. While temp→next ≠ NULL, move temp = temp→next

5. Set temp→prev→next = NULL
6. Free temp[1]

Polynomial Addition Algorithm

Steps:

1. Set pointer P1 to first polynomial, P2 to second polynomial
2. Create empty result polynomial
3. While P1 ≠ NULL AND P2 ≠ NULL:
    - If P1→exp > P2→exp:
        - Add P1 term to result
        - Move P1 = P1→next
    - Else if P2→exp > P1→exp:
        - Add P2 term to result
        - Move P2 = P2→next
    - Else (exponents equal):
        - Add (P1→coeff + P2→coeff) with exponent to result
        - Move P1 = P1→next and P2 = P2→next
4. Copy remaining terms from P1 (if any)
5. Copy remaining terms from P2 (if any)
6. Return result

Example: $5x^3+4x^2+2 + 3x^2+4x+5$

- Compare x^3: Only in P1 → Result: $5x^3$
- Compare x^2: Both have → Result: $5x^3 + 7x^2$
- Compare x^1: Only in P2 → Result: $5x^3 + 7x^2 + 4x$
- Compare x^0: Both have → Result: $5x^3 + 7x^2 + 4x + 7$[1]

3. Trees & BST

Preorder Traversal Algorithm (Root-Left-Right)

Steps:

1. If root == NULL, return
2. Visit (print) root node
3. Recursively traverse left subtree
4. Recursively traverse right subtree[1]

Inorder Traversal Algorithm (Left-Root-Right)

Steps:

1. If root == NULL, return
2. Recursively traverse left subtree
3. Visit (print) root node
4. Recursively traverse right subtree

Note: Produces sorted output for BST[1]

Postorder Traversal Algorithm (Left-Right-Root)

Steps:

1. If root == NULL, return
2. Recursively traverse left subtree
3. Recursively traverse right subtree
4. Visit (print) root node[1]

Example Tree:

```
     20
   10  30
 5  15
```

- Preorder: 20 10 5 15 30
- Inorder: 5 10 15 20 30 (Sorted!)
- Postorder: 5 15 10 30 20[1]

Tree Construction from Inorder + Preorder Algorithm

Steps:

1. Take first element from Preorder array → This is ROOT
2. Find ROOT position in Inorder array
3. Elements to left of ROOT in Inorder = Left subtree
4. Elements to right of ROOT in Inorder = Right subtree
5. Recursively build left subtree using corresponding elements
6. Recursively build right subtree using corresponding elements
7. Return ROOT

Example:

- Inorder: [D, B, E, A, F, C]

- Preorder: [A, B, D, E, C, F]

Steps:

1. First of Preorder = A → ROOT
2. Find A in Inorder: Left=[D,B,E], Right=[F,C]
3. Recursively process left and right[1]

BST Insert Algorithm

Steps:

1. If root == NULL:
   - Create new node with data
   - Return new node as root
2. If data < root→data:
   - Recursively insert into left subtree
   - Set root→left = result of recursion
3. Else if data > root→data:
   - Recursively insert into right subtree
   - Set root→right = result of recursion
4. Return root[1]

BST Search Algorithm

Steps:

1. If root == NULL, return false (not found)
2. If root→data == search_value, return true (found)
3. If search_value < root→data:
   - Recursively search left subtree
4. Else:
   - Recursively search right subtree

Time Complexity: O(log n) average[1]

AVL Tree Rotation Algorithms

LL Rotation (Single Right Rotation):
Steps:

1. Let unbalanced node be Z
2. Let left child of Z be Y
3. Perform right rotation at Z:

- Set Y→right = Z
- Set Z→left = previous Y→right

4. Update heights
5. Return Y as new root

Example:

```
    30 (BF=+2)          20
   /                   /  \
  20          →       10   30
 /
10
```

RR Rotation (Single Left Rotation):
Steps:

1. Let unbalanced node be Z
2. Let right child of Z be Y
3. Perform left rotation at Z:
   - Set Y→left = Z
   - Set Z→right = previous Y→left
4. Update heights
5. Return Y as new root[1]

LR Rotation (Left-Right Double Rotation):
Steps:

1. Perform left rotation at left child
2. Perform right rotation at root

Example:

```
  30               30              20
 /                /              /  \
10        →      20      →       10   30
  \             /
   20         10
```

RL Rotation (Right-Left Double Rotation):
Steps:

1. Perform right rotation at right child
2. Perform left rotation at root[1]

4. Graphs

DFS (Depth First Search) Algorithm

Steps:

1. Mark starting vertex as visited
2. Print/process the vertex
3. For each adjacent vertex of current vertex:
    - If not visited, recursively call DFS on that vertex
4. Continue until all reachable vertices are visited

Example: Graph: 0-1-3, 2 connected to 0 and 1

- DFS from 0: 0 → 1 → 3 → 2 (depth-first)[1]

BFS (Breadth First Search) Algorithm

Steps:

1. Create a queue and mark all vertices as not visited
2. Mark starting vertex as visited
3. Enqueue starting vertex
4. While queue is not empty:
    - Dequeue a vertex and print/process it
    - For each adjacent vertex of dequeued vertex:
        - If not visited, mark as visited and enqueue
5. Continue until queue is empty

Example: Same graph

- BFS from 0: 0 → 1 → 2 → 3 (level-wise)[1]

Prim's MST Algorithm

Steps:

1. Initialize: Select starting vertex (usually vertex 0)
2. Mark it as visited with cost = 0
3. Create MST set (initially empty)
4. Repeat (V-1) times:
    - Find minimum weight edge connecting visited vertex to unvisited vertex
    - Add this edge to MST
    - Mark the newly connected vertex as visited

- Add edge weight to total cost

5. Return MST and total cost

Example:

```
Graph:      2
        A-----B
        |1   /|5
        | 7/ |
        |/ 3 |
        C----F--2--J
```

Steps:

- Step 1: Start at A (cost=0)
- Step 2: Min edge from A: A-B(2) → Add to MST, cost=2
- Step 3: Min edge: A-C(1) → Add to MST, cost=3
- Step 4: Min edge: C-F(3) → Add to MST, cost=6
- Step 5: Min edge: F-J(2) → Add to MST, cost=8

Final: MST edges = {A-B, A-C, C-F, F-J}, Total cost = 8[1]

Kruskal's MST Algorithm

Steps:

1. Sort all edges in ascending order by weight
2. Create empty MST set
3. For each edge in sorted order:
   - If including this edge doesn't form a cycle:
     - Add edge to MST
     - Increment edge count
   - If edge count == V-1, stop
4. Return MST

Example:

- Edges sorted: (1,3)=1, (3,4)=2, (0,1)=3, (2,4)=4...
- Step 1: Add (1,3) with weight 1
- Step 2: Add (3,4) with weight 2
- Step 3: Add (0,1) with weight 3
- Continue until V-1 edges added[1]

Dijkstra's Shortest Path Algorithm

Steps:

1. Initialize dist[start] = 0, all others = ∞
2. Mark all vertices as unvisited
3. While there are unvisited vertices:
    - Select unvisited vertex u with minimum dist[u]
    - Mark u as visited
    - For each neighbor v of u:
        - Calculate new_dist = dist[u] + weight(u,v)
        - If new_dist < dist[v]:
            - Update dist[v] = new_dist
4. Return distance array

Example: Find shortest path from A:

```
Graph: A--1--B--2--D
       |2    |3
       C-----+
```

Result: dist[A]=0, dist[B]=1, dist[C]=2, dist[D]=3[1]

5. Sorting & Searching

Bubble Sort Algorithm

Steps:

1. For i = 0 to n-1:
    - For j = 0 to n-i-1:
        - If arr[j] > arr[j+1]:
            - Swap arr[j] and arr[j+1]
2. Array is now sorted

Example: Sort

- Pass 1: [68, 35, 48, 62, 52, 30, **98**] (largest at end)
- Pass 2: [35, 48, 62, 52, 30, **68**, 98]
- Pass 3: [35, 48, 52, 30, **62**, 68, 98]
- Continue until sorted:

Time Complexity: Best O(n), Average/Worst O(n^2), Stable[1]

Insertion Sort Algorithm

Steps:

1. For i = 1 to n-1:
    - Store arr[i] in key
    - Set j = i-1
    - While j ≥ 0 AND arr[j] > key:
        - Shift arr[j] to arr[j+1]
        - Decrement j
    - Place key at arr[j+1]
2. Array is now sorted

Example:

- Pass 1: [**17**, 68, 26, 54, 77, 93, 31] (insert 17)
- Pass 2: [17, **26**, 68, 54, 77, 93, 31] (insert 26)
- Pass 3: [17, 26, **54**, 68, 77, 93, 31]
- Continue...

Time Complexity: Best O(n), Average/Worst O(n^2), Stable[1]

Quick Sort Algorithm

Steps:

1. **Partition(arr, low, high):**
    - Select pivot = arr[high]
    - Set i = low-1
    - For j = low to high-1:
        - If arr[j] < pivot:
            - Increment i
            - Swap arr[i] and arr[j]
    - Swap arr[i+1] and arr[high]
    - Return i+1 (partition index)
2. **QuickSort(arr, low, high):**
    - If low < high:
        - pi = Partition(arr, low, high)
        - QuickSort(arr, low, pi-1)
        - QuickSort(arr, pi+1, high)

Example:

- Pivot = 25
- After partition: **25**
- Recursively sort left and right parts

Time Complexity: Best/Average O(n log n), Worst O(n²), Unstable[1]

Merge Sort Algorithm

Steps:

1. **Merge(arr, left, mid, right):**
    - Create temp arrays L and R
    - Copy data to L[0..n1] and R[0..n2]
    - Merge back to arr:
        - Compare elements from L and R
        - Place smaller element in arr
        - Copy remaining elements
2. **MergeSort(arr, left, right):**
    - If left < right:
        - Find mid = (left + right) / 2
        - MergeSort(arr, left, mid)
        - MergeSort(arr, mid+1, right)
        - Merge(arr, left, mid, right)

Time Complexity: Always O(n log n), Space O(n), Stable[1]

Radix Sort Algorithm (LSD to MSD)

Steps:

1. Find maximum number to determine number of digits
2. For each digit position (units, tens, hundreds...):
    - Perform counting sort based on current digit
    - Use stable sort to maintain relative order
3. After processing all digits, array is sorted

Example:

- Sort by units:
- Sort by tens:
- Sort by hundreds:

Time Complexity: O(d(n+k)) where d=digits, Stable[1]

Linear Search Algorithm

Steps:

1. For i = 0 to n-1:
    - If arr[i] == search_value:
        - Return i (index found)
2. Return -1 (not found)

Time Complexity: O(n)[1]

Binary Search Algorithm (Requires sorted array)

Steps:

1. Set left = 0, right = n-1
2. While left ≤ right:
    - Calculate mid = left + (right - left) / 2
    - If arr[mid] == search_value:
        - Return mid (found)
    - If arr[mid] < search_value:
        - Set left = mid + 1
    - Else:
        - Set right = mid - 1
3. Return -1 (not found)

Time Complexity: O(log n)
Example: 1000 elements → Linear: 1000 comparisons, Binary: 10 comparisons[1]

6. Hashing

Linear Probing Collision Resolution Algorithm

Formula: h(k,i) = (h(k) + i) % m, where i = 0,1,2...

Steps:

1. Calculate initial hash: index = h(k) = k % m
2. If table[index] is empty:
    - Insert key at table[index]
3. Else (collision):
    - Set i = 1
    - While table[(h(k) + i) % m] is not empty:

        - Increment i
    - Insert key at table[(h(k) + i) % m]

Example: Insert 18, 71, 42, 47 with h(k) = k % 8

- 18: h(18)=2 → table=18
- 71: h(71)=7 → table=71
- 42: h(42)=2 (collision!) → try (2+1)%8=3 → table=42
- 47: h(47)=7 (collision!) → try (7+1)%8=0 → table=47[1]

Quadratic Probing Algorithm

Formula: $h(k,i) = (h(k) + i^2) \% m$

Steps:

1. Calculate initial hash: index = h(k)
2. If collision occurs:
    - Try positions: $h(k)+1^2$, $h(k)+2^2$, $h(k)+3^2$... (all mod m)
3. Insert at first available position[1]

Double Hashing Algorithm

Formula: $h(k,i) = (h1(k) + i \times h2(k)) \% m$

Steps:

1. Define two hash functions:
    - h1(k) = k % m
    - h2(k) = k % (m-1) (must not be 0)
2. Calculate probe sequence using formula
3. Insert at first available position

Example: k=13, m=10

- h1(13)=3, h2(13)=13%9=4
- Probe sequence: 3, (3+4)%10=7, (3+8)%10=1...[1]

Separate Chaining Algorithm

Steps:

1. Create hash table as array of linked lists
2. To insert key k:
    - Calculate index = h(k) = k % m
    - Insert key at beginning of linked list at table[index]

3. To search key k:
    - Calculate index = h(k)
    - Search through linked list at table[index]

Example: Keys: 14,11,38,50,41,7,92,182,4 with h(k)=k%9

- Index 2: → → (chain)
- Index 5: → → (chain)[1]

7. Arrays & Memory

2D Array Row-Major Address Calculation Algorithm

Formula: Address(A[i][j]) = B + [(i-R1)×(C2-C1+1) + (j-C1)] × S

Steps:

1. Calculate number of rows: rows = R2-R1+1
2. Calculate number of columns: cols = C2-C1+1
3. Calculate row offset: row_offset = i-R1
4. Calculate column offset: col_offset = j-C1
5. Address = Base + [row_offset × cols + col_offset] × element_size

Example: A[10..15][10..40], find A, base=100, size=4

- Rows: 6, Cols: 31
- Address = 100 + [(15-10)×31 + (25-10)] × 4
- Address = 100 + [155 + 15] × 4 = 100 + 680 = **780**[1]

2D Array Column-Major Address Calculation Algorithm

Formula: Address(A[i][j]) = B + [(j-C1)×(R2-R1+1) + (i-R1)] × S

Steps:

1. Calculate number of rows: rows = R2-R1+1
2. Calculate number of columns: cols = C2-C1+1
3. Calculate column offset: col_offset = j-C1
4. Calculate row offset: row_offset = i-R1
5. Address = Base + [col_offset × rows + row_offset] × element_size

Same Example:

- Address = 100 + [(25-10)×6 + (15-10)] × 4
- Address = 100 + [90 + 5] × 4 = 100 + 380 = **480**[1]

8. Miscellaneous

Tower of Hanoi Algorithm

Steps:

1. **Base Case:** If n == 1:
   - Move disk 1 from source to destination
   - Return
2. **Recursive Case:**
   - Move n-1 disks from source to auxiliary (using destination)
   - Move nth disk from source to destination
   - Move n-1 disks from auxiliary to destination (using source)

Example: 3 disks (A→C using B)

1. Move 1: A→C
2. Move 2: A→B
3. Move 1: C→B
4. Move 3: A→C
5. Move 1: B→A
6. Move 2: B→C
7. Move 1: A→C

Total moves: $2^3-1 = 7$, **Time Complexity:** $O(2^n)$ [1]

This covers all the major algorithms from your PDF in proper algorithmic format without code!

⁂

1. RTU-DSA-EXAM-SURVIVAL-GUIDE-GUARANTEED-PASS-MA.pdf